

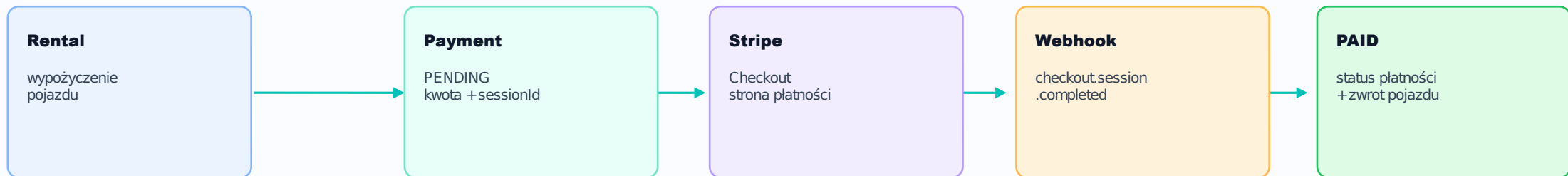
CarRent + Stripe: płatność online

Checkout Session, webhook, PaymentRepository, JPA adapter i testy lokalne

SPRING BOOT

STRIPE CHECKOUT

WEBHOOK



Backend tworzy sesję Stripe, zapisuje płatność jako PENDING, a status PAID ustawiamy po webhooku.

Stripe

<https://stripe.com/>

Stripe umożliwia przyjmowanie i przetwarzanie płatności przez internet. Wystarczy założyć konto i rozpocząć pracę ze środowiskiem testowym

Create your Stripe account

Email

asfafssg@mail.com

Full name

asfsafa

Password

Strong

Explore all products

For developers

Test mode

Publishable key

pk_test_51PT2RMKC1vWwDw...

Secret key

sk_test_51PT2RMKC1vWwDw...

View docs

- Home
- Balances
- Transactions
- Customers
- Product catalog
- Shortcuts
- Payments analytics
- Disputes
- Payment Links
- Invoices
- Products
- Payments
- Billing
- Reporting
- More

No code

Create a subscription from the Dashboard

Start

Create subscription

Jane Diaz

jane.x

\$3.00/unit/month

Metered usage

Options

Coupon

Tax

Trial

Billing

Charge payment method on file

Email invoices to customers to pay

Subscription Created

You successfully created a new subscription for Jane Diaz.

Add Trial

No code

Share a payment link

Start

Payment link is active

Share your payment link to accept payments.

buy.stripe.com/aEUa24H

Share

Today

Gross volume	Yesterday
zł2,100.00	zł4,800.00
12:49 PM	

PLN balance	View
zł8,420.18	
Estimated future payouts	

Stripe

Następnie należy dodać w projekcie nowe zmienne środowiskowe klucz api oraz klucz webhook.

Użyjemy je w ustawieniach (application.properties) Klucz api pobieramy ze strony:
<https://dashboard.stripe.com/test/apikeys>

Klucz dla webhook Wygenerujemy później za pomocą CLI.
Użyjemy tych zmiennych w application.properties:
stripe.api-key=\${STRIPE_API_KEY}
stripe.webhook-secret=\${WEBHOOK_SECRET}

Local listeners

+ Add local listener

DEVICE	VERSION	STATUS
 nitro localhost:8080/api/payments/webhook	1.27.0	Listening

Webhook to mechanizm wysyłania powiadomień o określonych zdarzeniach do aplikacji.
Pozwala na automatyczną reakcję na zdarzenia takie jak zakończenie płatności.
Za pomocą Stripe CLI możemy testować webhooki lokalnie.
CLI pobieramy ze strony: <https://github.com/stripe/stripe-cli/releases/tag/v1.27.0>
Klucz dla webhook uzyskamy po zalogowaniu (wpisujemy w konsoli)
./stripe.exe login

A następnie uruchamiamy nasłuchiwanie zdarzeń i przekazywanie do endpointu aplikacji:

```
luke@nitro MINGW64 ~/Desktop
$ ./stripe.exe listen --forward-to localhost:8080/api/payments/webhook
Ready! You are using Stripe API Version [2025-04-30.basil]. Your webhook signing secret is whsec_11f67a6b7806cca5abeca5b379fcdf8429cd3f6df2f04a03eed45f43d2e649ee (^C to quit)
```

Dodajemy klucz do zmiennych środowiskowych – WEBHOOK_SECRET
Przy każdym uruchomieniu nasłuchiwania zmienia się klucz!

Zakres zmian

Co dokładamy do istniejącego Spring Security + JWT

Przed zmianą

- /api/rentals/rent tworzy Rental
- /api/rentals/return kończy Rental
- Brak modelu płatności
- Brak Stripe Checkout
- Brak webhooka od dostawcy

Po zmianie

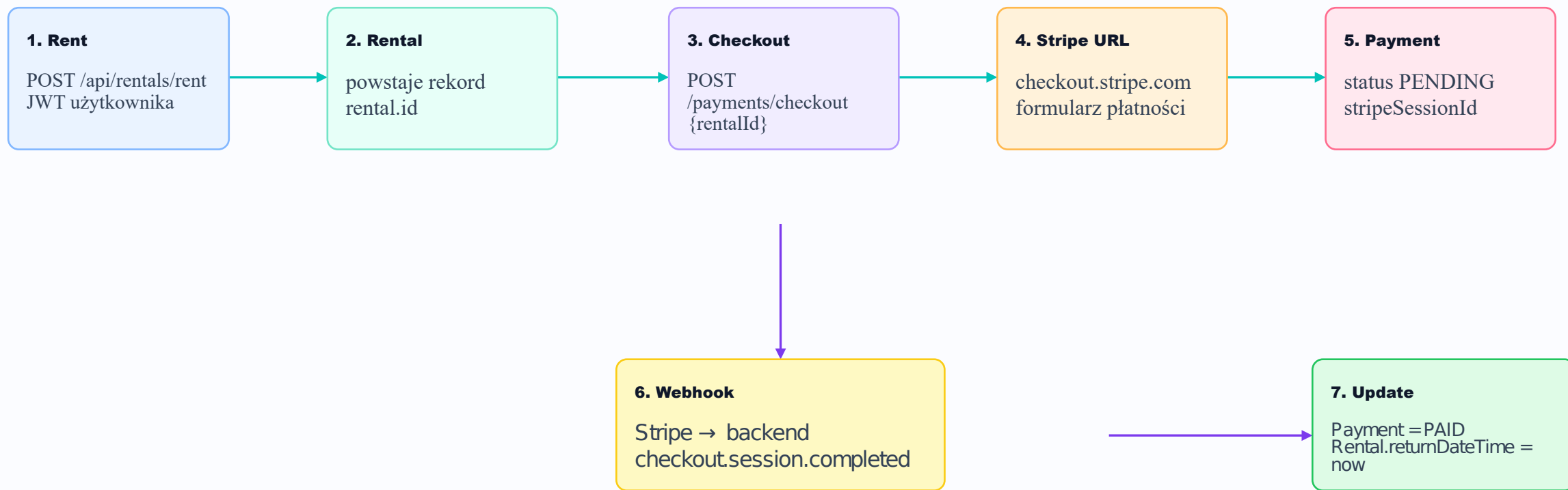
- Payment jako encja JPA
- PaymentStatus: PENDING / PAID
- Checkout Session w Stripe
- Webhook: checkout.session.completed
- Security: publiczny webhook

Założenia

- Stripe hostuje formularz płatności
- Backend trzyma stan transakcji
- Webhook jest źródłem prawdy
- Repozytorium ukrywa JPA
- Serwis trzyma logikę biznesową

Flow płatności

Od wypożyczenia pojazdu do statusu PAID



Zasada projektowa

Użytkownik może zostać przekierowany na /success, ale dopiero webhook potwierdza, że płatność została dokonana.

Stripe: konto, klucze i CLI

Sekrety trzymane w zmiennych środowiskowych, webhook testujemy lokalnie

ENV

```
STRIPE_API_KEY=sk_test_...  
WEBHOOK_SECRET=whsec_...  
STRIPE_SUCCESS_URL=http://localhost:8080/api/payments/success?session_id  
={CHECKOUT_SESSION_ID}  
STRIPE_CANCEL_URL=http://localhost:8080/api/payments/cancel
```

STRIPE CLI

```
stripe login  
stripe listen --forward-to localhost:8080/api/payments/webhook
```

API key

Klucz `sk\_test\_...` pozwala aplikacji tworzyć sesje Stripe Checkout. Nie commitujemy go do repozytorium.

Webhook secret

Sekret `whsec\_...` służy do weryfikacji podpisu Stripe-Signature. Zmienia się przy nowym listen.

Lokalny test

CLI odbiera event ze Stripe i przekazuje go do aplikacji na localhost:8080.

pom.xml: biblioteka Stripe

Bez SDK nie mamy klas Session, Webhook, Event ani SessionCreateParams

pom.xml

```
<dependency>
  <groupId>com.stripe</groupId>
  <artifactId>stripe-java</artifactId>
  <version>29.1.0</version>
</dependency>
```

Biblioteka Stripe

- Stripe.apiKey
- Session.create(params)
- Webhook.constructEvent(...)
- Event i Session jako obiekty Javy

application.yml: konfiguracja Stripe

Sekrety z ENV, URL-e z domyślnym fallbackiem lokalnym

application.yml

```
stripe:
  api-key: ${STRIPE_API_KEY}
  webhook-secret: ${WEBHOOK_SECRET}
  success-url: ${STRIPE_SUCCESS_URL:http://localhost:${PORT:8080}
/api/payments/success?session_id
={CHECKOUT_SESSION_ID}}
  cancel-url: ${STRIPE_CANCEL_URL:http://localhost:${PORT:8080}/api/payments/cancel}
```

api-key

Klucz prywatny Stripe. Backend używa go przy Session.create(...).

webhook-secret

Sekret do sprawdzenia podpisu requestu webhook.

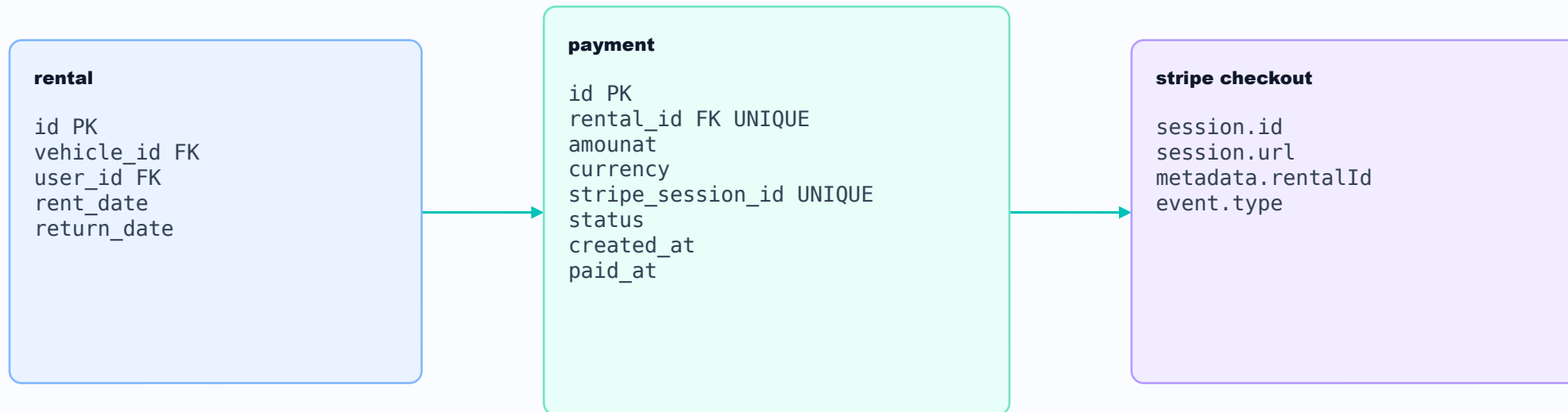
success/cancel

Adresy powrotu użytkownika po Stripe Checkout. Nie służą do oznaczania PAID.

Wartość w formacie `${ENV:default}` oznacza: użyj ENV, a jeśli go nie ma — użyj defaultu.

Model danych: Payment + Rental

Payment dopina płatność do konkretnego wypożyczenia



Relacja

@OneToOne: jedno wypożyczenie ma jedną płatność.
Kolumna rental_id jest w tabeli payment.

UNIQUE

Nie chcemy wielu płatności dla jednego wypożyczenia.
To pilnuje spójności danych.

sessionId

Webhook przychodzi ze Stripe z ID sesji. Po nim
odnajdujemy nasz Payment.

PaymentStatus

Statusy płatności jako enum zamiast dowolnego tekstu

PaymentStatus.java

```
package com.umcsuser.carrent.models;

public enum PaymentStatus {
    PENDING,
    PAID,
    CANCELED,
    FAILED
}
```

PENDING

Sesja Stripe została utworzona, ale webhook jeszcze nie potwierdził płatności.

PAID

Webhook checkout.session.completed przeszedł weryfikację i płatność jest opłacona.

CANCELED / FAILED

Przygotowanie na kolejne eventy Stripe.

Enum zapisujemy w encji jako @Enumerated(EnumType.STRING), żeby w bazie było PENDING/PAID, nie 0/1.

Encja Payment

Dane o płatności: kwota, Stripe Session ID, status i daty

Payment.java / pola główne

```
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
@EqualsAndHashCode(of = "id")
@ToString(exclude = "rental")
@Entity
@Table(name = "payment")
public class Payment {
    @Id
    @Column(nullable = false, unique = true)
    private String id;

    @OneToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "rental_id", nullable = false, unique = true)
    @JsonIgnore
    private Rental rental;

    @Column(nullable = false)
    private Long amount;
    @Column(nullable = false)
    private String currency;
}
```

Payment.java / status

```
@Column(name = "stripe_session_id", unique = true)
private String stripeSessionId;

@Enumerated(EnumType.STRING)
@Column(nullable = false)
private PaymentStatus status;

@Column(name = "created_at", nullable = false)
private String createdAt;

@Column(name = "paid_at")
private String paidAt;
```

@JsonIgnore

Chroni przed zapętlonym JSON-em:
Payment → Rental →
Payment → ...

Long amount

Stripe oczekuje groszy, np. 100
zł = 10000.

Rental: relacja @OneToOne

Wypożyczenie zna swoją płatność, a Payment jest właścicielem relacji

Rental.java

```
@OneToOne(  
    mappedBy = "rental",  
    cascade = CascadeType.ALL,  
    fetch = FetchType.EAGER  
)  
private Payment payment;
```

copy()

```
public Rental copy() {  
    return Rental.builder()  
        .id(id)  
        .vehicle(vehicle)  
        .user(user)  
        .rentDateTime(rentDateTime)  
        .returnDateTime(returnDateTime)  
        .payment(payment)  
        .build();  
}
```

mappedBy = "rental"

Kolumna rental_id znajduje się w tabeli payment. Rental nie jest właścicielem tej relacji.

cascade

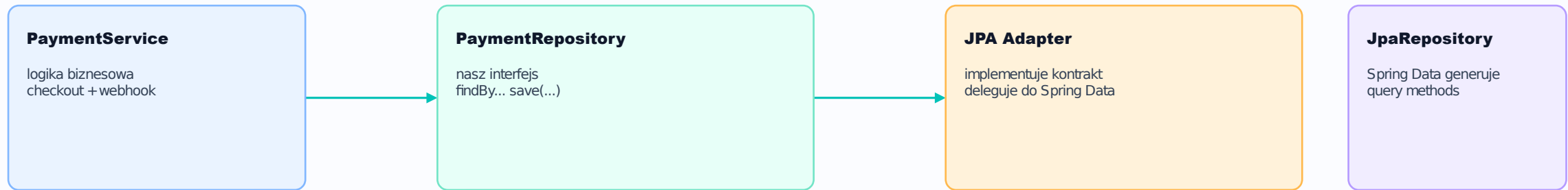
Gdy encje są powiązane, Hibernate rozumie zależność między Rental i Payment.

copy()

W projekcie JSON repozytoria używają copy(). Trzeba dopisać payment(payment)

Repozytoria: kontrakt + adapter JPA

Serwis nie zależy bezpośrednio od Spring Data



adapter

Serwis używa `PaymentRepository`, a nie `PaymentJpaRepository`. Dzięki temu architektura jest taka sama jak przy `RentalRepository`.

PaymentRepository

Wspólny kontrakt dla serwisu płatności

PaymentRepository.java

```
public interface PaymentRepository {  
    List<Payment> findAll();  
  
    Optional<Payment> findById(String id);  
  
    Optional<Payment> findByStripeSessionId(  
        String stripeSessionId);  
  
    Optional<Payment> findById(String rentalId);  
  
    Payment save(Payment payment);  
}
```

findByStripeSessionId

Potrzebne w webhooku. Stripe mówi: zakończono sesję cs_test_..., więc po tym ID szukamy Payment.

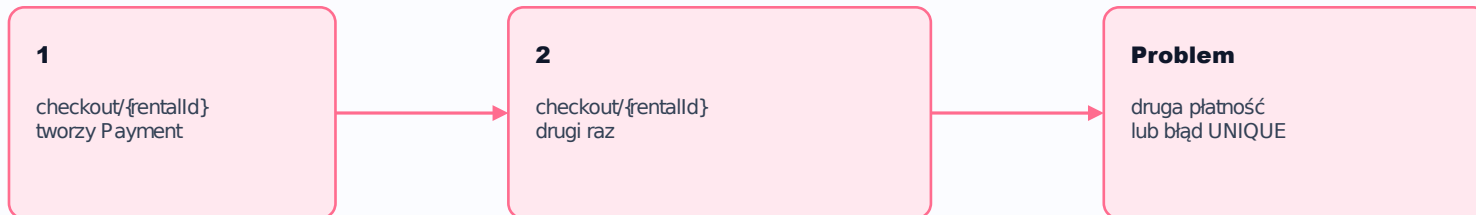
findById(String rentalId)

Checkout startuje od rentalId, więc sprawdzamy, czy płatność już istnieje.

save

Tworzy Payment PENDING albo zapisuje zmianę statusu na PAID po webhooku.

findByRentalId



Czy Payment istnieje?

Minimalnie wystarczy `findByStripeSessionId`, bo webhook szuka po sesji Stripe.

Projekt

Checkout wywołujemy po `rentalId`. Musimy sprawdzić, czy Payment dla Rental już istnieje.

Efekt

Jedno wypożyczenie = jedna płatność. Kod jest odporniejszy na podwójne kliknięcie.

bezpieczne tworzenie Payment

```
Payment payment = paymentRepository
    .findByRentalId(rental.getId())
    .orElseGet(() -> Payment.builder()
        .id(UUID.randomUUID().toString())
        .rental(rental)
        .createdAt(LocalDate.now().toString())
        .build());
```

PaymentJpaRepository

Spring Data JPA generuje zapytania z nazw metod

PaymentJpaRepository.java

```
@Profile("jpa")
public interface PaymentJpaRepository
    extends JpaRepository<Payment, String> {

    Optional<Payment> findByStripeSessionId(
        String stripeSessionId);

    Optional<Payment> findByRental_Id(
        String rentalId);
}
```

findByStripeSessionId

Pole istnieje bezpośrednio w Payment, więc nazwa metody jest prosta.

findByRental_Id

Payment ma pole rental, a rental ma pole id. Podkreślnik oznacza przejście do zagnieżdżonego pola.

findByRental_Id

```
// logicznie:
SELECT *
FROM payment
WHERE rental_id = ?;
```


PaymentRepositoryJpaAdapter

Adapter implementuje nasz interfejs i deleguje do Spring Data

adapter

```
@Repository
@Profile("jpa")
public class PaymentRepositoryJpaAdapter implements PaymentRepository {

    private final PaymentJpaRepository delegate;

    public PaymentRepositoryJpaAdapter(PaymentJpaRepository delegate) {
        this.delegate = delegate;
    }

    @Override
    public List<Payment> findAll() {
        return delegate.findAll();
    }

    @Override
    public Optional<Payment> findById(String id) {
        return delegate.findById(id);
    }

    @Override
    public Optional<Payment> findByStripeSessionId(String stripeSessionId) {
        return delegate.findByStripeSessionId(stripeSessionId);
    }

    @Override
    public Optional<Payment> findByRentalId(String rentalId) {
        return delegate.findByRental_Id(rentalId);
    }
}
```

Adapter cd. / save

```
public Payment save(Payment payment) {
    if (payment.getId() == null ||
        payment.getId().isBlank()) {
        payment.setId(UUID.randomUUID().toString());
    }
    return delegate.save(payment);
}
```

delegate

To właściwe Spring Data JPA repozytorium.
Adapter tylko przekazuje do niego wywołania.

UUID

Tak samo jak w Rental adapterze: jeśli encja
nie ma id, generujemy je przed save.

PaymentService: zależności i konfiguracja

Serwis trzyma logikę biznesową płatności

PaymentService.java

```
@Service
@Transactional
public class PaymentService
    implements PaymentServiceInterface {

    private final RentalRepository rentalRepository;
    private final PaymentRepository paymentRepository;

    @Value("${stripe.api-key}")
    private String apiKey;

    @Value("${stripe.webhook-secret}")
    private String webhookSecret;

    @Value("${stripe.success-url}")
    private String successUrl;

    @Value("${stripe.cancel-url}")
    private String cancelUrl;
}
```

@Service

Spring tworzy bean i pozwala wstrzyknąć serwis do PaymentController.

@Transactional

Zmiana Payment i Rental powinna być spójna: albo zapisze się całość, albo błąd przerwie transakcję.

RentalRepository

Szukamy wypożyczenia po rentalId i później zapisujemy returnDateTime.

PaymentRepository

Tworzymy PENDING i po webhooku aktualizujemy na PAID.

@Value

Sekrety i URL-e nie są wpisane w kodzie. Pochodzą z application.yml i ENV.

createCheckoutSession: początek

Najpierw sprawdzamy wypożyczenie i wyliczamy kwotę dla Stripe

createCheckoutSession / walidacja

```
@Override
public String createCheckoutSession(String rentalId) {
    Rental rental = rentalRepository.findById(rentalId)
        .orElseThrow(() -> new IllegalArgumentException(
            "Nie znaleziono wypożyczenia."));

    if (!rental.isActive()) {
        throw new IllegalStateException(
            "To wypożyczenie zostało już zakończone.");
    }

    Stripe.apiKey = apiKey;
    long amount = Math.round(
        rental.getVehicle().getPrice() * 100);
}
```

findById

Checkout startuje od rentalId, więc najpierw znajdujemy wypożyczenie.

isActive()

Nie tworzymy płatności dla zakończonego wypożyczenia.

Stripe.apiKey

Bez klucza API biblioteka Stripe nie wykona requestu do konta testowego.

amount * 100

Stripe przyjmuje kwotę w groszach: 300 zł → 30000.

createCheckoutSession: produkt i cena

Stripe Checkout potrzebuje pozycji koszyka: produkt, kwota, ilość

ProductData + PriceData

```
SessionCreateParams.LineItem.PriceData.ProductData productData =  
    SessionCreateParams.LineItem.PriceData.ProductData.builder()  
        .setName("Zwrot pojazdu: "  
            + rental.getVehicle().getBrand()  
            + " "  
            + rental.getVehicle().getModel())  
        .build();  
  
SessionCreateParams.LineItem.PriceData priceData =  
    SessionCreateParams.LineItem.PriceData.builder()  
        .setCurrency("pln")  
        .setUnitAmount(amount)  
        .setProductData(productData)  
        .build();
```

LineItem

```
SessionCreateParams.LineItem lineItem =  
    SessionCreateParams.LineItem.builder()  
        .setQuantity(1L)  
        .setPriceData(priceData)  
        .build();
```

ProductData

Tekst widoczny na stronie płatności Stripe.

PriceData

Waluta + kwota w groszach + opis produktu.

createCheckoutSession: SessionCreateParams

Tu mówimy Stripe, jak ma wyglądać sesja płatności

parametry sesji

```
SessionCreateParams params =  
    SessionCreateParams.builder()  
        .setMode(SessionCreateParams.Mode.PAYMENT)  
        .setSuccessUrl(successUrl)  
        .setCancelUrl(cancelUrl)  
        .addLineItem(lineItem)  
        .putMetadata("rentalId", rental.getId())  
        .build();  
  
Session session = Session.create(params);
```

Mode.PAYMENT

Jednorazowa płatność. Nie subskrypcja i nie samo zapamiętanie karty.

successUrl / cancelUrl

Adresy powrotu użytkownika po płatności lub anulowaniu.

metadata.rentalId

Ułatwia debugowanie w Stripe Dashboard. W bazie i tak trzymamy stripeSessionId.

Session.create(params) wykonuje request do Stripe API i zwraca session.getUrl(), czyli link do strony płatności.

createCheckoutSession: zapis Payment PENDING

Po utworzeniu sesji zapisujemy lokalny stan płatności

zapis stanu przed płatnością

```
Payment payment = paymentRepository.findByRentalId(rental.getId())
    .orElseGet(() -> Payment.builder()
        .id(UUID.randomUUID().toString())
        .rental(rental)
        .createdAt(LocalDateTime.now().toString())
        .build());

payment.setAmount(amount);
payment.setCurrency("pln");
payment.setStripeSessionId(session.getId());
payment.setStatus(PaymentStatus.PENDING);

paymentRepository.save(payment);
return session.getUrl();
```

findByRentalId

Chroni przed duplikatem, gdy ktoś ponownie wywoła checkout dla tego samego Rental.

stripeSessionId

Najważniejszy identyfikator do późniejszego webhooka.

PENDING

Sesja istnieje, ale płatność nie jest jeszcze potwierdzona.

session.getUrl()

To link, który frontend albo Postman otworzy w przeglądarce.

handleWebhook: weryfikacja podpisu

Webhook musi być publiczny, ale nie może ufać każdemu requestowi

handleWebhook / podpis

```
@Override
public void handleWebhook(String payload, String signature) {
    Stripe.apiKey = apiKey;

    Event event;

    try {
        event = Webhook.constructEvent(
            payload,
            signature,
            webhookSecret);
    } catch (SignatureVerificationException e) {
        throw new IllegalArgumentException(
            "Nieprawidłowy podpis Stripe webhook.");
    }
}
```

payload

JSON wysłane przez Stripe.

signature

Nagłówek Stripe-Signature z requestu.

webhookSecret

Sekret whsec_... z CLI/Dashboard, znany tylko naszej aplikacji i Stripe.

Weryfikacja!

Bez weryfikacji ktoś mógłby udawać Stripe i ręcznie oznaczyć płatność jako PAID.

handleWebhook: event checkout.session.completed

Obsługujemy tylko zdarzenie zakończonego Checkouta

event → Session → Payment

```
if (!"checkout.session.completed".equals(event.getType())) {
    return;
}

Session session = (Session) event.getDataObjectDeserializer()
    .getObject()
    .orElseThrow(() -> new IllegalStateException(
        "Nie udało się odczytać sesji Stripe."));

String stripeSessionId = session.getId();

Payment payment = paymentRepository
    .findByStripeSessionId(stripeSessionId)
    .orElseThrow(() -> new IllegalStateException(
        "Nie znaleziono płatności dla sesji Stripe."));
```

event.getType()

Stripe wysyła wiele typów zdarzeń. Nas interesuje zakończony checkout.

Session

Obiekt sesji zawiera ID, które zapisaliśmy wcześniej w Payment.

findByStripeSessionId

webhook identyfikuje płatność po sesji Stripe.

handleWebhook: PAID i zwrot pojazdu

Po potwierdzeniu płatności aktualizujemy Payment i Rental

status i domena

```
if (payment.getStatus() == PaymentStatus.PAID) {
    return;
}

payment.setStatus(PaymentStatus.PAID);
payment.setPaidAt(LocalDateTime.now().toString());
paymentRepository.save(payment);

Rental rental = payment.getRental();

if (rental.isActive()) {
    rental.setReturnDateTime(LocalDateTime.now().toString());
    rentalRepository.save(rental);
}
```

Idempotencja

Webhook może przyjść więcej niż raz.
Jeśli Payment już jest PAID, nic nie robimy.

Payment = PAID

To jest właściwe potwierdzenie płatności. Nie robimy tego w success URL.

Rental.returnDateTime

W tej wersji po opłaceniu kończymy wypożyczenie/zwrot pojazdu.

PaymentController

REST API dla checkoutu, webhooka i redirectów

PaymentController.java

```
@RestController
@RequestMapping("/api/payments")
public class PaymentController {
    private final PaymentServiceInterface paymentService;

    @PostMapping("/checkout/{rentalId}")
    public ResponseEntity<Map<String, String>> createCheckoutSession(
        @PathVariable String rentalId) {
        String url = paymentService.createCheckoutSession(rentalId);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(Map.of("url", url));
    }

    @PostMapping("/webhook")
    public ResponseEntity<Void> handleWebhook(
        @RequestBody String payload,
        @RequestHeader("Stripe-Signature") String signature) {
        paymentService.handleWebhook(payload, signature);
        return ResponseEntity.ok().build();
    }
}
```

/checkout/{rentalId}

Endpoint chroniony JWT. Użytkownik tworzy URL do płatności dla swojego Rental.

/webhook

Endpoint publiczny, ale zabezpieczony podpisem Stripe-Signature.

redirect endpoints

```
@GetMapping("/success")
return ResponseEntity.ok("success");

@GetMapping("/cancel")
return ResponseEntity.ok("cancel");
```

SecurityConfig: publiczny webhook

Stripe nie ma tokena J WT, więc webhook musi przejść bez logowania

SecurityConfig.java

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .requestMatchers("/health").permitAll()
    .requestMatchers("/api/payments/webhook").permitAll()
    .requestMatchers("/api/payments/success").permitAll()
    .requestMatchers("/api/payments/cancel").permitAll()
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/users/**").hasRole("ADMIN")
    .anyRequest().authenticated()
)
```

Webhook publiczny

Stripe wysła POST bez Authorization Bearer. Gdyby endpoint wymagał J WT, dostalibyśmy 401/403.

Webhook nadal bezpieczny

Bezpieczeństwo przenosimy na podpis Stripe-Signature i webhookSecret.

Checkout chroniony

Nie dodaliśmy permitAll dla /checkout/{rentalId}. Płatność tworzy zalogowany użytkownik.

Testy: Postman + Stripe CLI

Najpierw JWT, potem rental, checkout, płatność i webhook

1. Login

POST /api/auth/login
→ kopiujemy token J WT

2. Rental

POST /api/rentals/rent
Authorization: Bearer <token>
{ "vehicleId": "1" }

3. Checkout

POST /api/payments/checkout/{rentalId}
→ dostajemy url

4. Stripe

Otwieramy url
płacimy kartą testową
Stripe wysyła webhook

terminal

```
stripe login  
stripe listen --forward-to localhost:8080/api/payments/webhook
```

Postman

```
POST /api/payments/checkout/884646be-...  
Authorization: Bearer <TOKEN>  
  
{  
  "url": "https://checkout.stripe.com/c/pay/cs_test_..."  
}
```

Po checkout.session.completed: payment.status = PAID, payment.paid_at != null, rental.return_date != null.

Checklist implementacji

1. pom.xml: stripe-java
2. application.yml: api-key, webhook-secret, success-url, cancel-url
3. PaymentStatus enum
4. Payment jako @Entity
5. Rental.payment jako @OneToOne
6. PaymentRepository + findByStripeSessionId + findByRentalId
7. PaymentJpaRepository + findByRental_Id
8. PaymentRepositoryJpaAdapter
9. PaymentService: createCheckoutSession + handleWebhook
10. PaymentController: checkout, webhook, success, cancel
11. SecurityConfig: permitAll dla webhook/success/cancel
12. Test: login → rent → checkout → Stripe CLI → PAID

Podsumowanie: Stripe obsługuje stronę płatności, a nasza aplikacja odpowiada za stan domeny: Payment i Rental.